

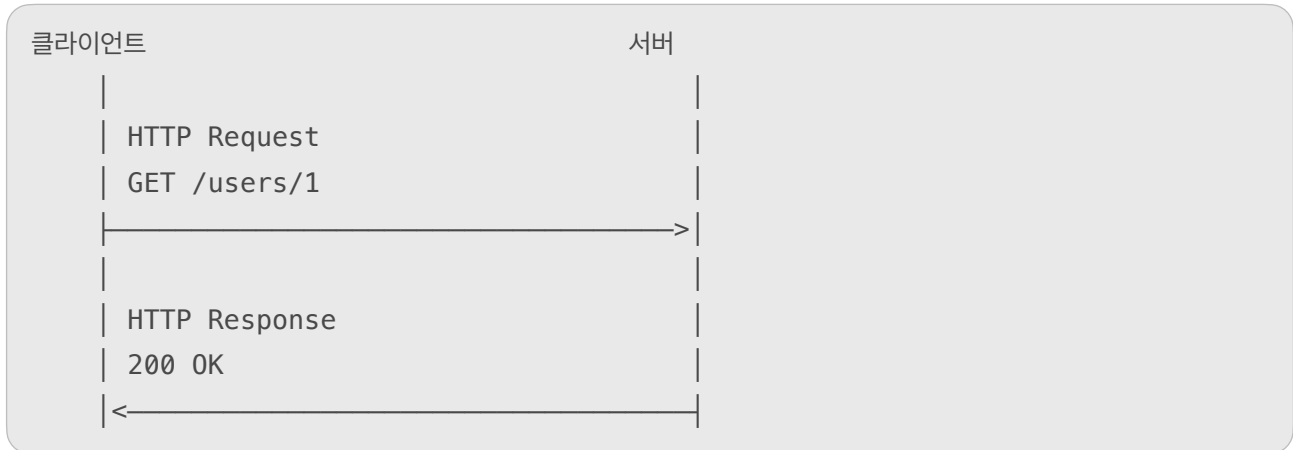
# 네트워크 5 - HTTP

남궁명수

## [ HTTP ]

HTTP(Hypertext Transfer Protocol)는 클라이언트와 서버가 웹 데이터를 주고받기 위해 사용하는 **애플리케이션 계층 프로토콜**이다.

HTTP는 기본적으로 요청(Request)과 응답(Response) 방식으로 동작한다.



예를 들어 브라우저가 서버에 사용자 정보를 요청하면:

```
GET /users/1 HTTP/1.1
Host: example.com
Accept: application/json
```

서버는 다음과 같이 응답한다.

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 27

{"id":1,"name":"Myeongsu"}
```

---

## [1] [ HTTP의 특징 ]

### [ 요청과 응답 구조 ]

HTTP 통신은 클라이언트가 요청을 보내고 서버가 응답을 반환하는 구조다.

```
클라이언트: 어떤 작업을 원하는지 요청
서버: 요청 처리 결과를 응답
```

HTTP 요청에는 일반적으로 다음 정보가 들어간다.

HTTP Method  
요청 대상 URI  
HTTP Version  
Header  
Body

HTTP 응답에는 다음 정보가 들어간다.

HTTP Version  
Status Code  
Reason Phrase  
Header  
Body

### [ Stateless ]

HTTP는 기본적으로 이전 요청의 상태를 기억하지 않는 **Stateless 프로토콜**이다.

첫 번째 요청: 로그인  
두 번째 요청: 사용자 정보 요청

HTTP 프로토콜 자체만 보면 서버는 두 요청이 같은 사용자에게서 왔는지 자동으로 기억하지 않는다.

로그인 상태를 유지하려면 다음과 같은 별도의 정보를 사용한다.

- Cookie
- Session
- JWT
- Authorization 헤더

중요한 점은 Keep-Alive와 로그인 상태 유지가 다르다는 것이다.

Keep-Alive  
→ TCP 연결을 재사용  
  
Cookie, Session, JWT  
→ 사용자 인증 및 애플리케이션 상태 유지

## [2] [ HTTP 요청 구조 ]

HTTP/1.1 요청은 개념적으로 다음과 같이 구성된다.

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 35
Authorization: Bearer token-value

{"name":"Myeongsu","age":28}
```

### [시작 줄]

```
POST /users HTTP/1.1
```

| 항목       | 의미          |
|----------|-------------|
| POST     | HTTP Method |
| /users   | 요청 대상       |
| HTTP/1.1 | HTTP 버전     |

### [Header]

요청에 대한 부가 정보를 전달한다.

```
Host: example.com
Content-Type: application/json
Authorization: Bearer token-value
```

대표적인 요청 헤더는 다음과 같다.

| 헤더             | 의미                   |
|----------------|----------------------|
| Host           | 요청 대상 서버의 호스트 이름     |
| Content-Type   | 요청 본문의 데이터 형식        |
| Content-Length | 요청 본문의 바이트 길이        |
| Accept         | 클라이언트가 받을 수 있는 응답 형식 |
| Authorization  | 인증 정보                |
| Cookie         | 클라이언트가 저장한 쿠키        |
| User-Agent     | 요청을 보낸 프로그램 정보       |

### [Body]

서버에 전달할 실제 데이터를 담는다.

```
{
  "name": "Myeongsu",
  "age": 28
}
```

주로 POST, PUT, PATCH 등의 요청에서 사용한다.

### [3] [ HTTP 응답 구조 ]

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 27
```

```
{"id":1,"name":"Myeongsu"}
```

### [상태 줄]

```
HTTP/1.1 200 OK
```

| 항목       | 의미           |
|----------|--------------|
| HTTP/1.1 | HTTP 버전      |
| 200      | Status Code  |
| OK       | 상태 코드에 대한 설명 |

### [응답 Header]

```
Content-Type: application/json
Content-Length: 27
```

대표적인 응답 헤더는 다음과 같다.

| 헤더             | 의미                    |
|----------------|-----------------------|
| Content-Type   | 응답 본문의 데이터 형식         |
| Content-Length | 응답 본문의 길이             |
| Set-Cookie     | 클라이언트에 쿠키 저장 요청       |
| Location       | 생성된 자원 위치 또는 리다이렉트 위치 |
| Cache-Control  | 캐시 정책                 |
| Server         | 서버 소프트웨어 정보           |

### [응답 Body]

클라이언트에게 전달하는 실제 데이터다.

```
{
  "id": 1,
  "name": "Myeongsu"
}
```

204 No Content나 HEAD 요청처럼 응답 Body가 없는 경우도 있다.

### [4] [ HTTP Method ]

HTTP Method는 클라이언트가 서버의 자원에 대해 어떤 작업을 원하는지 표현한다.  
대표적인 Method는 다음과 같다.

| Method  | 주요 목적            |
|---------|------------------|
| GET     | 자원 조회            |
| POST    | 자원 생성 또는 작업 요청   |
| PUT     | 자원 전체 교체         |
| PATCH   | 자원 일부 수정         |
| DELETE  | 자원 삭제            |
| HEAD    | Body 없이 응답 정보 조회 |
| OPTIONS | 지원하는 통신 옵션 확인    |

### [GET]

서버의 자원을 조회한다.

```
GET /users/1 HTTP/1.1
Host: example.com
```

예를 들어:

```
GET /users      → 사용자 목록 조회
GET /users/1    → 1번 사용자 조회
GET /posts/10   → 10번 게시물 조회
```

일반적으로 조회 조건은 Query Parameter로 전달할 수 있다.

```
GET /users?page=1&size=20
```

GET 요청은 서버 상태를 변경하지 않는 조회 용도로 설계해야 한다.

```
GET /users/1/delete
```

처럼 GET으로 데이터를 삭제하도록 설계하면 안 된다. 브라우저의 미리 불러오기, 검색 엔진 크롤링, 캐시 등의 동작만으로 의도하지 않은 변경이 발생할 수 있다.

### [POST]

새로운 자원을 생성하거나 특정 작업을 요청할 때 사용한다.

```
POST /users HTTP/1.1
Content-Type: application/json

{
  "name": "Myeongsu"
}
```

서버가 새로운 사용자를 생성하면 다음과 같이 응답할 수 있다.

```
HTTP/1.1 201 Created
Location: /users/10
Content-Type: application/json

{
  "id": 10,
  "name": "Myeongsu"
}
```

Location 헤더는 생성된 자원의 위치를 알려준다.

POST는 자원 생성뿐 아니라 단순 CRUD로 표현하기 어려운 작업에도 사용될 수 있다.

```
POST /orders/10/pay
POST /capsules/10/open
POST /users/10/login
```

POST 요청은 일반적으로 멍등성을 보장하지 않는다.

동일한 생성 요청을 두 번 보내면 자원이 두 개 생성될 수 있다.

```
POST /orders → 주문 1 생성
POST /orders → 주문 2 생성
```

### [PUT]

특정 URI의 자원을 요청 Body의 내용으로 **전체 교체**할 때 주로 사용한다.

```
PUT /users/10 HTTP/1.1
Content-Type: application/json

{
  "name": "Myeongsu",
  "age": 28,
  "email": "user@example.com"
}
```

기존 자원:

```
{
  "name": "Kim",
  "age": 20,
  "email": "old@example.com"
}
```

PUT 이후:

```
{
  "name": "Myeongsu",
  "age": 28,
  "email": "user@example.com"
}
```

PUT은 일반적으로 자원의 전체 상태를 전달하며, 여러 번 동일하게 요청해도 최종 결과가 같다.

### [PATCH]

기존 자원의 일부만 수정할 때 사용한다.

```
PATCH /users/10 HTTP/1.1
Content-Type: application/json

{
  "name": "Myeongsu"
}
```

기존 자원:

```
{
  "name": "Kim",
  "age": 20,
  "email": "old@example.com"
}
```

PATCH 이후:

```
{
  "name": "Myeongsu",
  "age": 20,
  "email": "old@example.com"
}
```

PUT과 PATCH의 차이를 단순화하면 다음과 같다.

```
PUT    → 자원 전체 교체
PATCH → 자원 일부 수정
```

### [DELETE]

지정한 자원을 삭제한다.

```
DELETE /users/10 HTTP/1.1
```

응답은 다음과 같이 구성할 수 있다.

```
HTTP/1.1 204 No Content
```

또는 삭제된 자원의 정보를 반환한다면:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "deleted": true
}
```



## [HEAD]

GET과 유사하지만 응답 Body를 받지 않는다.

```
HEAD /large-file.zip HTTP/1.1
```

응답:

```
HTTP/1.1 200 OK
Content-Type: application/zip
Content-Length: 104857600
```

Body를 다운로드하지 않고 다음 내용을 확인할 때 사용할 수 있다.

- 자원 존재 여부
- 파일 크기
- 수정 시간
- 캐시 정보
- Content-Type

## [OPTIONS]

서버 또는 특정 자원이 지원하는 통신 옵션을 확인한다.

```
OPTIONS /users HTTP/1.1
```

응답 예시:

```
HTTP/1.1 204 No Content
Allow: GET, POST, OPTIONS
```

브라우저의 CORS Preflight 요청에도 OPTIONS가 사용된다.

```
브라우저
| OPTIONS 요청
▼
서버가 허용된 Origin, Method, Header 응답
|
▼
조건이 허용되면 실제 요청 전송
```

## [5] [ 안전성과 멍등성 ]

HTTP Method를 이해하려면 **안전성(Safe)** 과 **멍등성(Idempotent)** 을 구분해야 한다.

### [안전한 메서드]

요청이 서버의 자원 상태를 변경하기 위한 것이 아니면 안전하다고 한다.  
대표적인 안전한 메서드는 다음과 같다.

GET  
HEAD  
OPTIONS

예를 들어 GET 요청이 서버 로그나 조회 횟수를 남기는 부수 효과까지 절대 없어야 한다는 뜻은 아니다.  
요청의 주된 목적이 자원 변경이 아니라 조회라는 의미다.

### [멍등한 메서드]

같은 요청을 한 번 보내든 여러 번 보내든 서버 자원의 **최종 상태가 같은 성질**이다.  
예를 들어 같은 PUT 요청을 반복하면:

PUT 1회 → 사용자 이름이 Myeongsu  
PUT 2회 → 사용자 이름이 Myeongsu  
PUT 3회 → 사용자 이름이 Myeongsu

최종 상태가 같으므로 멍등하다.  
DELETE도 멍등하다.

DELETE 1회 → 자원 삭제  
DELETE 2회 → 이미 없음  
DELETE 3회 → 이미 없음

각 요청의 응답 상태 코드는 다를 수 있지만 최종적으로 자원이 존재하지 않는다는 상태는 같다.

반면 POST로 주문을 생성하면:

POST 1회 → 주문 1개 생성  
POST 2회 → 주문 1개 추가 생성  
POST 3회 → 주문 1개 추가 생성

최종 상태가 달라질 수 있으므로 일반적으로 멍등하지 않다.

| Method  | 안전성     | 멍등성           |
|---------|---------|---------------|
| GET     | 안전      | 멍등            |
| HEAD    | 안전      | 멍등            |
| OPTIONS | 안전      | 멍등            |
| PUT     | 안전하지 않음 | 멍등            |
| DELETE  | 안전하지 않음 | 멍등            |
| POST    | 안전하지 않음 | 일반적으로 멍등하지 않음 |
| PATCH   | 안전하지 않음 | 멍등성이 보장되지 않음  |

멍등성은 응답 내용이 항상 같다는 뜻이 아니라 서버 자원의 최종 상태가 같다는 뜻이다.

## [6] [ Status Code ]

HTTP Status Code는 서버가 클라이언트의 요청을 처리한 결과를 나타내는 세 자리 숫자다. 첫 번째 숫자에 따라 의미가 나뉜다.

| 범위  | 분류            | 의미                |
|-----|---------------|-------------------|
| 1xx | Informational | 요청 처리 중           |
| 2xx | Success       | 요청 성공             |
| 3xx | Redirection   | 추가 이동 또는 캐시 사용 필요 |
| 4xx | Client Error  | 클라이언트 요청에 문제      |
| 5xx | Server Error  | 서버 처리 중 문제        |

## [7] [ 1xx: 처리 중 ]

서버가 요청을 받았으며 아직 처리가 진행 중임을 나타낸다.

### [100 Continue]

클라이언트가 큰 요청 Body를 보내기 전에 서버가 이를 받을 준비가 되었는지 확인할 때 사용한다.

Expect: 100-continue

### [101 Switching Protocols]

서버가 클라이언트의 프로토콜 전환 요청을 수락했음을 나타낸다.

예를 들어 HTTP/1.1 연결을 WebSocket으로 업그레이드할 때 사용할 수 있다.

## [8] [ 2xx: 요청 성공 ]

### [200 OK]

요청이 정상적으로 처리되었다.

GET 성공  
POST 작업 성공  
PATCH 수정 성공

HTTP/1.1 200 OK

### [201 Created]

요청으로 새로운 자원이 성공적으로 생성되었다.

HTTP/1.1 201 Created  
Location: /users/10

주로 POST 요청의 성공 응답에 사용한다.

### [202 Accepted]

서버가 요청을 접수했지만 아직 처리가 완료되지 않았다.

대용량 파일 처리  
이메일 발송 요청  
백그라운드 작업  
비동기 작업 큐 등록

HTTP/1.1 202 Accepted

이는 최종 작업 성공을 의미하지 않고, 처리 요청을 받아들였다는 의미다.

### [204 No Content]

요청은 성공했지만 응답 Body가 없다.

HTTP/1.1 204 No Content

자원 삭제나 응답 데이터가 필요 없는 수정 요청 등에 사용할 수 있다.

---

### [9] [ 3xx: 리다이렉션과 캐시 ]

#### [301 Moved Permanently]

자원의 주소가 영구적으로 변경되었다.

```
HTTP/1.1 301 Moved Permanently
Location: https://new.example.com
```

검색 엔진이나 클라이언트는 앞으로 새로운 주소를 사용할 수 있다.

#### [302 Found]

자원이 일시적으로 다른 위치에 있음을 나타낸다.

```
HTTP/1.1 302 Found
Location: /login
```

로그인되지 않은 사용자를 로그인 페이지로 이동시킬 때 자주 볼 수 있다.

#### [304 Not Modified]

클라이언트가 가진 캐시가 여전히 유효하므로 응답 Body를 다시 보내지 않는다.

```
클라이언트:
“내가 가진 파일 이후로 수정됐나요?”

서버:
“수정되지 않았으니 기존 캐시를 사용하세요.”
```

304는 오류가 아니라 캐시를 활용하기 위한 정상적인 응답이다.

#### [307 Temporary Redirect]

임시 리다이렉션이며 기존 HTTP Method와 Body를 유지해야 한다.

```
POST 요청 → 307 → 이동한 주소에도 POST 요청
```

#### [308 Permanent Redirect]

영구 리다이렉션이며 기존 HTTP Method와 Body를 유지해야 한다.

| 코드  | 이동 성격 | Method 유지             |
|-----|-------|-----------------------|
| 301 | 영구    | 클라이언트 동작에 따라 변경될 수 있음 |
| 302 | 임시    | 클라이언트 동작에 따라 변경될 수 있음 |
| 307 | 임시    | 유지                    |
| 308 | 영구    | 유지                    |

## [10] [ 4xx: 클라이언트 오류 ]

클라이언트가 보낸 요청에 문제가 있음을 나타낸다.

### [400 Bad Request]

요청 형식이나 값이 잘못되어 서버가 요청을 처리할 수 없다.

잘못된 JSON

필수 필드 누락

잘못된 데이터 형식

예를 들어:

```
{  
  "age": "문자열"  
}
```

서버가 age에 숫자를 기대했다면 400을 반환할 수 있다.

### [401 Unauthorized]

이름은 Unauthorized지만 실질적으로는 인증되지 않았거나 인증 정보가 유효하지 않은 상태에 사용한다.

Authorization 헤더 없음

JWT 만료

잘못된 로그인 정보

쉽게 말하면:

“누구인지 확인되지 않았습니다.”

### [403 Forbidden]

인증은 되었지만 해당 작업을 수행할 권한이 없다.

일반 사용자가 관리자 API 요청

다른 사용자의 비공개 자원 접근

쉽게 말하면: “누구인지는 알지만 이 작업을 수행할 권한이 없습니다.”

401 → 인증 문제: 누구인지 확인되지 않음

403 → 인가 문제: 누구인지는 알지만 권한이 없음

### [404 Not Found]

요청한 자원을 찾을 수 없다.

GET /users/999

→ 999번 사용자가 존재하지 않음

보안상 자원의 존재를 감추기 위해 권한이 없는 사용자에게 404를 반환하는 경우도 있다.

#### [405 Method Not Allowed]

자원은 존재하지만 요청한 HTTP Method를 지원하지 않는다.

/users는 존재  
DELETE /users는 허용하지 않음

#### [409 Conflict]

요청이 현재 서버의 자원 상태와 충돌한다.

이미 사용 중인 이메일  
이미 처리된 초대 수락  
동시에 수정하여 버전 충돌

#### [415 Unsupported Media Type]

서버가 요청 Body의 데이터 형식을 지원하지 않는다.

Content-Type: application/xml

서버가 JSON만 지원한다면 415를 반환할 수 있다.

#### [422 Unprocessable Content]

요청 형식 자체는 이해했지만 내용상 검증에 실패해 처리할 수 없다.

이메일 형식 오류  
비밀번호 길이 부족  
open\_at이 현재보다 이전 날짜

예를 들어 FastAPI에서 요청 Body의 필드 타입이나 검증 조건이 맞지 않을 때 422 응답을 자주 볼 수 있다.

#### [429 Too Many Requests]

클라이언트가 일정 시간 동안 너무 많은 요청을 보냈다.

API Rate Limit 초과  
로그인 시도 횟수 초과

서버는 재시도 가능 시간을 알려줄 수 있다.

Retry-After: 60

### [11] [ 5xx: 서버 오류 ]

요청 자체보다는 서버가 처리하는 과정에서 문제가 발생했음을 나타낸다.

#### [500 Internal Server Error]

서버 내부에서 예상하지 못한 오류가 발생했다.

처리되지 않은 예외  
Nil Pointer 오류  
데이터베이스 처리 오류  
프로그램 버그

#### [501 Not Implemented]

서버가 요청받은 기능이나 Method를 구현하지 않았다.

#### [502 Bad Gateway]

게이트웨이나 프록시 서버가 뒤쪽 서버로부터 잘못된 응답을 받았다.

클라이언트  
↓  
Nginx  
↓  
Backend 서버 연결 또는 응답 문제

예를 들어 네가 겪었던 모바일 카카오 로그인 502는 Nginx 같은 중간 서버가 Backend로부터 정상적인 응답을 받지 못했을 가능성을 확인해야 하는 상태 코드다.

#### [503 Service Unavailable]

서버가 일시적으로 요청을 처리할 수 없다.

서버 과부하  
점검 중  
일시적인 자원 부족

#### [504 Gateway Timeout]

게이트웨이나 프록시가 뒤쪽 서버의 응답을 정해진 시간 안에 받지 못했다.

클라이언트  
↓  
Nginx  
↓  
Backend 응답 지연  
↓  
Timeout



## [502와 504의 차이]

502 Bad Gateway

→ 중간 서버가 Backend에서 유효한 응답을 받지 못함

504 Gateway Timeout

→ 중간 서버가 Backend 응답을 제한 시간 안에 받지 못함

---

## [12] [ HTTP Keep-Alive ]

HTTP Keep-Alive는 하나의 TCP 연결을 여러 HTTP 요청과 응답에 재사용하는 기능이다.  
Keep-Alive를 사용하지 않으면 요청마다 TCP 연결을 새로 만들어야 한다.

### [Keep-Alive를 사용하지 않는 경우]

TCP 연결 설정

↓

HTTP 요청 1

↓

HTTP 응답 1

↓

TCP 연결 종료

TCP 연결 설정

↓

HTTP 요청 2

↓

HTTP 응답 2

↓

TCP 연결 종료

요청마다 다음 비용이 반복된다.

- TCP 3-Way Handshake
- TCP 연결 상태 생성
- 연결 종료
- 추가적인 네트워크 왕복 시간
- HTTPS라면 TLS Handshake 비용

웹페이지 하나에도 HTML, CSS, JavaScript, 이미지 등 여러 자원이 필요하다.

```
GET /index.html
GET /style.css
GET /app.js
GET /logo.png
```

요청마다 새 TCP 연결을 만들면 비효율적이다.

#### [Keep-Alive를 사용하는 경우]

```
TCP 연결 설정
↓
HTTP 요청 1 → 응답 1
↓
HTTP 요청 2 → 응답 2
↓
HTTP 요청 3 → 응답 3
↓
TCP 연결 종료
```

하나의 TCP 연결을 여러 요청에 재사용하므로 연결 설정 비용을 줄일 수 있다.

```
TCP 연결 1개
├ GET /index.html
├ GET /style.css
├ GET /app.js
└ GET /logo.png
```

---

### [13] [ HTTP 버전과 Keep-Alive ]

#### [HTTP/1.0]

HTTP/1.0에서는 일반적으로 요청과 응답이 끝나면 TCP 연결을 종료했다.  
연결을 유지하려면 명시적으로 다음 헤더를 사용할 수 있었다.

```
Connection: keep-alive
```

#### [HTTP/1.1]

HTTP/1.1에서는 지속 연결이 기본 동작이다.  
따라서 일반적으로 다음 헤더를 매번 지정할 필요가 없다.

```
Connection: keep-alive
```

서버나 클라이언트가 연결을 종료하고 싶으면 다음을 사용한다.

```
Connection: close
```

정리하면:

HTTP/1.0

- 기본적으로 연결 종료
- Keep-Alive를 원하면 명시

HTTP/1.1

- 기본적으로 지속 연결
- 종료하려면 Connection: close

다만 HTTP/1.1이라고 해서 연결이 영원히 유지되는 것은 아니다.

서버는 다음 이유로 연결을 종료할 수 있다.

- Keep-Alive Timeout 만료
- 최대 요청 횟수 초과
- 서버 자원 부족
- 애플리케이션 종료
- 네트워크 오류

---

#### [14] [ Keep-Alive Timeout ]

서버는 마지막 HTTP 요청 이후 일정 시간 동안 다음 요청이 오지 않으면 TCP 연결을 종료할 수 있다.

HTTP 요청·응답 완료

↓

일정 시간 대기

↓

추가 요청 없음

↓

TCP 연결 종료

예를 들어 개념적으로 다음처럼 설정할 수 있다.

```
timeout = 5초  
max requests = 100
```

이는 하나의 연결을 최대 100개 요청까지 재사용하고, 5초 동안 다음 요청이 없으면 종료한다는 의미다. 실제 설정 방식은 Nginx, Apache, Go HTTP 서버 등 서버 구현에 따라 다르다.

---

### [15] [ Keep-Alive 와 TCP Keepalive의 차이 ]

이름은 비슷하지만 서로 다른 기능이다.

#### [HTTP Keep-Alive]

하나의 TCP 연결을 여러 HTTP 요청에 재사용한다.

목적: TCP 연결 설정 비용 절감

#### [TCP Keepalive]

오랫동안 데이터 교환이 없는 TCP 연결에서 상대방이 여전히 살아 있는지 확인하기 위해 Probe 패킷을 보내는 기능이다.

목적: 끊어진 TCP 연결 탐지

| 구분 | HTTP Keep-Alive       | TCP Keepalive   |
|----|-----------------------|-----------------|
| 계층 | 애플리케이션 계층             | 전송 계층           |
| 목적 | 연결 재사용                | 죽은 연결 탐지        |
| 대상 | HTTP 요청과 응답           | TCP 연결 자체       |
| 동작 | 여러 HTTP 요청을 같은 연결로 전송 | 유향 연결에 Probe 전송 |

### [16] [ Keep-Alive와 로그인 상태의 차이 ]

Keep-Alive는 사용자 로그인 상태를 유지하는 기능이 아니다.

Keep-Alive

→ 물리적으로 같은 TCP 연결을 재사용

Cookie/Session/JWT

→ 사용자가 누구인지 확인

Keep-Alive 연결이 종료되더라도 Cookie나 JWT가 있다면 새로운 TCP 연결에서 로그인 상태를 이어갈 수 있다.

반대로 TCP 연결이 유지되고 있어도 인증 정보가 없다면 서버가 해당 사용자를 로그인된 사용자로 판단할 수 없다

### [17] [ HTTP/2와 HTTP/3 ]

예를 들어, 웹페이지에 다음 파일들이 있다고 가정한다.

index.html

style.css

app.js

image.png

## [HTTP/1.1]

TCP 연결 하나에서는 일반적으로 요청과 응답을 하나씩 순서대로 처리한다.

TCP 연결 1

HTML 요청 → HTML 응답 완료

CSS 요청 → CSS 응답 완료

JS 요청 → JS 응답 완료

이미지 요청 → 이미지 응답 완료

TCP 연결을 재사용할 수는 있지만, 앞의 응답을 처리하는 동안 뒤의 요청이 기다려야 해서 느릴 수 있다.

그래서 브라우저는 TCP 연결을 여러 개 만들어 병렬 처리한다.

TCP 연결 1 → HTML

TCP 연결 2 → CSS

TCP 연결 3 → JavaScript

TCP 연결 4 → 이미지

즉, HTTP/1.1은 여러 개의 TCP 연결을 이용해 여러 파일을 동시에 받는 방식이 주로 사용되었다.

- 한 계산대에서 한 명씩 처리
- 빠르게 하려고 계산대를 여러 개 만들

## [HTTP/2]

HTTP/2는 TCP 연결을 여러 개 만드는 대신, TCP 연결 하나를 여러 개의 논리적인 통신 스트림으로 나눠서 사용한다.

즉, 하나의 TCP 연결 안에서 여러 요청과 응답을 독립적인 스트림으로 동시에 처리하는 Multiplexing을 지원한다.

TCP 연결 하나

├ Stream 1: HTML 요청·응답

├ Stream 3: CSS 요청·응답

├ Stream 5: JavaScript 요청·응답

└ Stream 7: 이미지 요청·응답

실제로는 데이터를 작은 프레임으로 나눠 섞어서 전송한다.

HTML 일부

CSS 일부

이미지 일부

HTML 나머지

JavaScript 일부

CSS 나머지

각 데이터에는 스트림 번호가 있어서 브라우저가 다시 분류될 수 있다.

Stream 1 데이터 → HTML로 조립  
Stream 3 데이터 → CSS로 조립  
Stream 5 데이터 → JavaScript로 조립

- 계산대 하나가 여러 손님의 물건을 조금씩 번갈아 처리

### [HTTP/3]

HTTP/3도 여러 스트림을 동시에 사용한다는 점은 HTTP/2와 비슷하다.  
차이는 TCP 대신 UDP 위에서 동작하는 QUIC을 사용한다는 것이다.

QUIC 연결 하나

Stream 1 → HTML  
Stream 2 → CSS  
Stream 3 → JavaScript  
Stream 4 → 이미지

HTTP/2에서는 TCP 차원에서 데이터가 유실되면 다른 스트림까지 함께 기다릴 수 있다.

HTML 데이터 유실  
→ TCP 복구 대기  
→ CSS, JS, 이미지 데이터도 영향을 받을 수 있음

HTTP/3의 QUIC은 스트림을 독립적으로 관리해서 특정 스트림의 데이터가 유실되더라도 다른 스트림은 계속 처리할 수 있다.

HTML 데이터 유실  
→ HTML 스트림만 복구 대기  
  
CSS 스트림 → 계속 처리  
JS 스트림 → 계속 처리  
이미지 스트림 → 계속 처리

단, HTTP/3가 UDP를 사용한다고 해서 신뢰성이 없는 것은 아니다.

QUIC이 UDP 위에서 재전송, 순서 보장, 흐름 제어, 암호화 같은 기능을 직접 제공한다.

- 여러 손님이 독립적으로 처리되어 한 손님의 문제가 다른 손님을 덜 막음

### [18] [ curl로 HTTP 확인하기 ]

요청과 응답 헤더를 자세히 확인하려면 다음 명령을 사용할 수 있다.

```
curl -v http://example.com
```

대략 다음 정보가 출력된다.

```
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/...
> Accept: */*

< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: ...
```

응답 헤더만 확인하려면:

```
curl -I https://example.com
```

POST 요청은 다음처럼 보낼 수 있다.

```
curl -v \
  -X POST \
  -H 'Content-Type: application/json' \
  -d '{"name": "Myeongsu"}' \
  http://localhost:8080/users
```

참고로 -d를 사용하면 curl은 기본적으로 POST 요청을 사용하므로 보통 -X POST는 생략할 수 있다.

---

### [19] [ TCP와 HTTP의 관계 ]

HTTP는 애플리케이션이 사용하는 통신 규칙이고, TCP는 데이터를 신뢰성 있게 전달하는 전송 계층 프로토콜이다.

HTTP/1.1을 기준으로 전체 과정은 다음과 같다.

1. 클라이언트가 socket() 생성
2. connect()로 서버에 TCP 연결 요청
3. TCP 3-Way Handshake
4. HTTP 요청 전송
5. 서버가 HTTP 응답 전송
6. Keep-Alive라면 TCP 연결 유지
7. 같은 연결로 추가 HTTP 요청·응답
8. 더 이상 사용하지 않으면 TCP 연결 종료

HTTP:

GET /users/1

200 OK

Content-Type: application/json

TCP:

HTTP 메시지를 순서대로 손실 없이 전달

HTTP와 TCP의 역할은 다르다.

HTTP → 무엇을 요청하고 어떤 결과를 반환하는가?

TCP → 해당 데이터를 어떻게 신뢰성 있게 전달하는가?

IP → 어느 컴퓨터까지 전달하는가?